



# Bases de datos distribuidas

## Modelo ACID

M. Elena Rodríguez González  
Jordi Conesa i Caralt

**EIMT**.UOC.EDU

Bienvenidos a una nueva presentación del bloque temático dedicado a bases de datos distribuidas. En ella vamos a tratar los principales problemas que se derivan del acceso simultáneo de diversos usuarios a una misma base de datos y cómo éstos pueden ser tratados.

## Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

EIMT.UOC.EDU

Uno de los objetivos fundamentales de los sistemas gestores de bases de datos es garantizar la consistencia de los datos. Existen diferentes situaciones que pueden comprometerla. Una de ellas es el acceso simultáneo de los usuarios a un recurso compartido, la base de datos. Recordemos que ésta puede ser centralizada o distribuida.

Para tratar los accesos concurrentes, los sistemas gestores de bases de datos usan transacciones e implementan mecanismos que las gestionan. Uno de los modelos de transacciones más relevante es el modelo ACID, que es el foco de interés de esta presentación. Dicho modelo nos servirá de base para entender el modelo BASE que utilizan algunas bases de datos NoSQL.

# Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

A continuación vamos a revisar el concepto de consistencia y las situaciones que pueden conducir a que se vea comprometida.

## Consistencia

- Uno de los objetivos más importantes de un SGBD es garantizar la consistencia de los datos.
- La consistencia tiene que ver con la integridad, calidad y corrección de los datos, y se puede ver comprometida en diferentes situaciones:
  - Errores de programación
  - Violación de restricciones de integridad
  - El acceso simultáneo de diferentes usuarios
  - La existencia de datos replicados
  - Averías

EIMT.UOC.EDU

La consistencia es un concepto amplio y con múltiples acepciones. Se relaciona con la integridad, la calidad y la corrección de los datos. Como ya hemos comentado, los sistemas gestores de base de datos tienen que velar por su cumplimiento.

Hay diversas causas que pueden comprometerla, siendo las más destacables la posibilidad de que se produzcan errores de programación, el hecho que las operaciones puedan incumplir las restricciones de integridad definidas, el acceso simultáneo de los usuarios a la base de datos, la existencia de réplicas y la posibilidad de que ocurran averías.

## Consistencia

- Las averías pueden ser de naturaleza diversa:
  - Caídas del SGBD debidas, por ejemplo, a una avería software o un corte de luz
  - La destrucción total o parcial de la BD a causa de desastres o de fallos de los dispositivos de almacenamiento
  - Fallos en la red de comunicaciones que pueden causar el particionamiento del sistema.
  - Pérdida de mensajes
- Para garantizar la consistencia de la BD el SGBD se basa, principalmente, en transacciones y en una serie de mecanismos que las gestionan.

EIMT.UOC.EDU

Las averías pueden ser de naturaleza diversa y de distinto nivel de gravedad, aunque todas ellas comparten el hecho de que, además de la consistencia, también se ve comprometida la disponibilidad de la base de datos, es decir, la capacidad de responder a las peticiones que formulan las aplicaciones y/o los usuarios.

Ejemplos de averías serían la caída del sistema gestor de la base de datos o la destrucción total o parcial del contenido de la base de datos (ésta puede ser centralizada o distribuida). También hay que añadir la posibilidad de que se produzcan fallos en la red que causen el particionamiento del sistema. Esto es especialmente relevante en una base de datos distribuida, dado que puede motivar que algunos nodos no estén accesibles. Finalmente, también se pueden perder mensajes (que se usan a efectos de coordinación) entre los diferentes nodos que forman la base de datos distribuida o que éstos no sean entregados a tiempo.

Para garantizar la consistencia, el sistema gestor de la base de datos (sea centralizado o distribuido) se basa, principalmente, en transacciones y en una serie de mecanismos que las gestionan. También se usan dietarios y copias de seguridad de la base de datos.

# Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

EIMT.UOC.EDU

A continuación vamos a explicar el modelo de transacciones ACID. Para ello, en primer lugar, presentaremos el concepto de transacción. A continuación veremos sus propiedades.

El modelo ACID data de principios de la década de los 80 y es el resultado del trabajo desarrollado por diversos investigadores. Entre ellos destaca Jim Gray que recibió el premio Alan Turing en 1998 por sus contribuciones al área de bases de datos y procesamiento de transacciones, y por su liderazgo en la implementación de sistemas gestores de bases de datos. Por su parte, el acrónimo ACID fue acuñado en 1983 por Andreas Reuter y Theo Härder.

# Transacciones y propiedades ACID

Una transacción es un conjunto de operaciones (de lectura y/o escritura) sobre la BD que se ejecutan como una unidad indivisible de trabajo. La transacción acaba su ejecución confirmando o cancelando los cambios que se han llevado a cabo sobre la BD.

- Las transacciones deben cumplir las propiedades ACID:
  - **Atomicidad:** el conjunto de operaciones que constituyen la transacción es la unidad indivisible de ejecución.
  - **Consistencia:** la ejecución de la transacción tiene que preservar la consistencia de la BD.
  - **Aislamiento:** una transacción no puede ver interferida su ejecución por otras transacciones que se estén ejecutando de forma concurrente con ella.
  - **Definitividad/Durabilidad:** los resultados producidos por una transacción que confirma tienen que ser definitivos en la BD.

EIMT.UOC.EDU

Una transacción constituye un conjunto de operaciones sobre la base de datos que se ejecutan como una unidad indivisible. Las operaciones incluidas en la transacción, desde un punto de vista lógico, están relacionadas entre sí. Es por ello que la transacción constituye la unidad indivisible de ejecución.

Una transacción puede finalizar su ejecución de dos formas: confirmando sus resultados (para ello ejecuta una petición de *commit*) o cancelándolos (en este caso, la última petición de la transacción sería una operación de *rollback*).

Las transacciones deben cumplir una serie de propiedades que se conocen bajo el acrónimo ACID. Estas propiedades son: atomicidad, consistencia, aislamiento y definitividad (o durabilidad). La atomicidad indica que el conjunto de operaciones que constituyen la transacción es la unidad atómica (o indivisible) de ejecución. Esto quiere decir que nunca queda a medias, es decir, o bien se ejecutan todas las operaciones que incluye (y, en este caso, la transacción confirma sus resultados) o bien no se ejecuta ninguna en absoluto (y, en este caso, los cancela).

La ejecución de una transacción tiene que preservar la consistencia de la base de datos. En otras palabras, si antes de ejecutarse una transacción la base de datos se encuentra en un estado consistente (es decir, en un estado donde se verifican todas las reglas de integridad), al acabar su ejecución la base de datos también debe estar en un estado consistente. Durante la ejecución de la transacción, se acepta que momentáneamente la base de datos pueda caer en un estado inconsistente. Si la base de datos es distribuida, es necesario mantener la consistencia de las réplicas que de unos mismos datos puedan existir.

También es necesario garantizar el correcto aislamiento de las transacciones, es decir, una transacción no puede ver interferida su ejecución por ninguna otra transacción que se esté ejecutando de forma concurrente con ella.

Finalmente, los resultados producidos por una transacción que confirma tienen que ser definitivos en la base de datos, es decir, no se pueden perder, incluso en el caso que se produzcan fallos o desastres, hasta que otra transacción cambie esos resultados y los confirme. Por el contrario, los resultados producidos por una transacción que aborta su ejecución se tienen que descartar de la base de datos.

Las propiedades ACID no son independientes entre sí. Por ejemplo, la atomicidad y definitividad están estrechamente interrelacionadas. Por su parte, la consistencia se relaciona con todas ellas. Asimismo destacar que, el cumplimiento de las propiedades ACID de las transacciones, es una responsabilidad compartida entre el sistema gestor de la base de datos y los desarrolladores de las aplicaciones.

## Ejemplo de transacción

| Transferencia de cantidad Q de cuenta_origen a cuenta_destino |                                                                                                                                                                                                           |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N.º operación                                                 | Operaciones que hay que ejecutar                                                                                                                                                                          |
| 1                                                             | saldo_origen:= leer_saldo(cuenta_origen)<br>Comprobar que saldo_origen es mayor o igual a Q<br>(suponemos que hay saldo suficiente para hacer la transferencia)                                           |
| 2                                                             | saldo_destino:= leer_saldo(cuenta_destino)                                                                                                                                                                |
| 3                                                             | escribir_saldo(cuenta_origen, saldo_origen - Q)                                                                                                                                                           |
| 4                                                             | escribir_saldo(cuenta_destino, saldo_destino + Q)                                                                                                                                                         |
| 5                                                             | registrar_movimiento("transferencia", cuenta_origen, cuenta_destino, Q)<br>Crear un registro para anotar la transferencia en una tabla de movimientos,<br>y poner también la fecha y la hora, por ejemplo |

EIMT.UOC.EDU

Esta transparencia muestra uno de los ejemplos más clásicos (el de la transferencia bancaria) que se utiliza para ilustrar el concepto de transacción.

En él podemos ver, en pseudocódigo, las operaciones incluidas en la transacción, y el orden en que se ejecutan. Al realizar una transferencia, como clientes de una entidad bancaria, esperamos que todas (o ninguna) de las operaciones se lleven a cabo (atomicidad).

En el caso que la transferencia se realice, esperamos que los saldos de las cuentas implicadas contengan los valores correctos de acuerdo a la petición que hemos efectuado (consistencia y definitividad).

Por el contrario, en caso de problemas, esperamos que los saldos de las cuentas sean los que existían antes del inicio de la ejecución de la transacción (atomicidad y consistencia). Por ejemplo, si justo después de ejecutar la operación 3 (donde ya se ha restado la cantidad a transferir al saldo de la cuenta origen) se produce una avería que impide que el proceso finalice, esperamos que la entidad bancaria garantice que el saldo de la cuenta origen no se vea alterado, dado que la cantidad a transferir no se ha hecho efectiva en la cuenta de destino.

Asimismo, aunque pueda parecer una obviedad, esperamos (y también lo espera la entidad bancaria) que la lógica de la aplicación asociada a la transferencia esté bien codificada (consistencia). De forma similar, si otros clientes están utilizando los servicios del banco, esperamos que sus peticiones no interfieran con las nuestras, y mucho menos produzcan resultados anómalos (aislamiento y consistencia). En otras palabras, como clientes, queremos tener la percepción de que únicamente nosotros estamos trabajando con la aplicación (y en consecuencia con la base de datos que almacena nuestros datos).



# Interferencias

- La propiedad de aislamiento se puede ver vulnerada debido a la presencia de interferencias.
- Existen diferentes tipos de interferencias:
  - Actualización perdida
  - Lectura no confirmada
  - Lectura no repetible
  - Análisis inconsistente (fantasmas)

EIMT.UOC.EDU

La propiedad de aislamiento se puede ver vulnerada debido a la presencia de interferencias. Existen 4 grandes tipos de interferencias, de diferente nivel de gravedad.

Antes de entrar en su análisis, es necesario tener claras las condiciones que pueden conducir a su existencia.

En primer lugar, al menos dos de las transacciones acceden a una misma porción de la base de datos. En segundo lugar, como mínimo, una de las dos transacciones, sobre esta porción común a la que acceden, efectúa operaciones de actualización.

En otras palabras, cuando las transacciones que se ejecutan de forma concurrente sólo hacen lecturas, nunca se producirán interferencias. De manera similar, en caso de que las transacciones hagan actualizaciones, si éstas se realizan sobre porciones diferentes de la base de datos, tampoco se pueden producir interferencias.

A continuación vamos a explicar los diferentes tipos de interferencias que se pueden producir.

## Actualización perdida

cuenta: 500€

| Transacción T1 (reintegro de 20)               | Transacción T2 (reintegro de 40)               |
|------------------------------------------------|------------------------------------------------|
| saldo:= leer_saldo(cuenta) <b>500€</b>         |                                                |
|                                                | saldo:= leer_saldo(cuenta) <b>500€</b>         |
| escribir_saldo(cuenta, saldo – 20) <b>480€</b> |                                                |
|                                                | escribir_saldo(cuenta, saldo – 40) <b>460€</b> |
| COMMIT                                         |                                                |
|                                                | COMMIT                                         |

cuenta: 460€

EIMT.UOC.EDU

La interferencia de actualización perdida (en inglés, *lost update* o *write-write*) se produce porque se efectúan dos lecturas consecutivas de un mismo dato seguidas de dos cambios consecutivos del mismo dato, tal y como muestra el ejemplo.

T1 y T2 son dos transacciones que efectúan reintegros (de 20 y 40€ respectivamente) de una misma cuenta bancaria. Las dos transacciones leen el mismo valor del saldo de la cuenta (500€), y lo actualizan de forma independiente, restando al saldo leído la cantidad a reintegrar.

Suponiendo que el sistema gestor de la base de datos ejecuta las operaciones de cada transacción sin ningún control y en el orden que se propone, el cambio correspondiente a la transacción T1 se pierde debido a que ha visto interferida su ejecución por la transacción T2. En consecuencia, el saldo de la cuenta disminuye en 40€, en vez de en 60€.

A pesar que desde la perspectiva de quienes efectúan los reintegros se han “ganado” 20€, desde el punto de vista del banco se han “perdido” 20€, hecho que reviste una “cierta” gravedad. El saldo final de la cuenta es erróneo (o en otras palabras, inconsistente), dado que debería ser de 440€ en lugar de 460€. Aunque el ejemplo quizá pueda despertar nuestra “simpatía”, podríamos haber planteado situaciones donde el perjudicado hubiera sido el cliente (imaginemos el caso que T1 fuera la transacción que ingresa nuestra nómina).

## Lectura no confirmada

| cuenta: 500€                    |                                         |
|---------------------------------|-----------------------------------------|
| Transacción T1 (consulta saldo) | Transacción T2 (reintegro de 20)        |
|                                 | saldo:= leer_saldo(cuenta) 500€         |
|                                 | escribir_saldo(cuenta, saldo - 20) 480€ |
| saldo:= leer_saldo(cuenta) 480€ |                                         |
| COMMIT                          |                                         |
|                                 | ROLLBACK                                |
| cuenta: 500€                    |                                         |

| cuenta: 500€                    |                                         |
|---------------------------------|-----------------------------------------|
| Transacción T1 (consulta saldo) | Transacción T2 (dos reintegros de 50)   |
|                                 | saldo:= leer_saldo(cuenta) 500€         |
|                                 | escribir_saldo(cuenta, saldo - 50) 450€ |
| saldo:= leer_saldo(cuenta) 450€ |                                         |
| COMMIT                          |                                         |
|                                 | saldo:= leer_saldo(cuenta) 450€         |
|                                 | escribir_saldo(cuenta, saldo - 50) 400€ |
|                                 | COMMIT                                  |
| cuenta: 400€                    |                                         |

EIMT.UOC.EDU

La interferencia de lectura no confirmada (*dirty read* en inglés) se puede producir cuando una transacción recupera un dato pendiente de confirmación que ha sido modificado por otra transacción que se ejecuta de forma concurrente con ella. Esta situación se puede producir básicamente en dos escenarios.

El primero se relaciona con la cancelación de transacciones, tal y como muestra el ejemplo de la parte superior izquierda. En él podemos ver la ejecución concurrente de dos transacciones: una de consulta de saldo de una cuenta y otra que intenta efectuar un reintegro de esa misma cuenta. En el ejemplo, la transacción T2 lee el saldo de la cuenta y lo disminuye en la cantidad que se quiere reintegrar. A continuación, la transacción T1 efectúa la consulta de saldo. El valor de saldo de 480€ que obtiene T1 está pendiente de confirmar, es un dato provisional, ya que T2 todavía no ha confirmado. Acto seguido, la transacción T1 finaliza su ejecución y confirma los resultados. Finalmente T2 cancela su ejecución. Esta cancelación causa que los resultados sean descartados, de manera que el saldo de la cuenta pasa a ser el que había antes de empezar la ejecución de T2. En consecuencia, T1 ha recuperado un valor que oficialmente nunca ha existido y ha visto interferida su ejecución por la transacción T2.

Por su parte, el segundo escenario se relaciona con la reescritura de un mismo dato. Esta situación se muestra en el ejemplo de la parte inferior derecha. De nuevo, suponiendo que el sistema gestor de la base de datos no realiza ningún control sobre el orden de ejecución de las operaciones, podemos ver que, a pesar de que la transacción T2 confirma sus resultados, T1 ve interferida su ejecución por T2 dado que recupera un valor intermedio. Este valor corresponde a un saldo provisional para la cuenta corriente que corresponde al que queda después del primer reintegro.

Destacar que, a pesar de que se ha producido una interferencia, el saldo final de la cuenta es correcto. En otras palabras, la situación reviste menor gravedad que en el caso de la interferencia de actualización perdida.

## Lectura no repetible

cuenta: 500€

| Transacción T1 (reintegro de 20)   |      | Transacción T2 (consulta saldo) |      |
|------------------------------------|------|---------------------------------|------|
|                                    |      | saldo:= leer_saldo(cuenta)      | 500€ |
| saldo:= leer_saldo(cuenta)         | 500€ |                                 |      |
| escribir_saldo(cuenta, saldo - 20) | 480€ |                                 |      |
|                                    |      | saldo:= leer_saldo(cuenta)      | 480€ |
| COMMIT                             |      |                                 |      |
|                                    |      | COMMIT                          |      |

cuenta: 480€

EIMT.UOC.EDU

La interferencia de lectura no repetible (en inglés *nonrepeatable read*) se produce cuando una transacción, por los motivos que sea, necesita leer dos veces un mismo dato y en cada lectura recupera un valor diferente debido a que existe otra transacción que se ejecuta simultáneamente con ella y que efectúa una modificación del dato leído, tal y como se puede apreciar en el ejemplo.

Nuevamente, suponiendo que el sistema gestor de la base de datos ejecuta las operaciones que constituyen cada transacción sin ningún control y en el orden que se propone, podemos ver que la transacción T2 en cada consulta del saldo de la cuenta corriente, recupera un valor diferente (de 500 y 480€ respectivamente) por el hecho de que la transacción T1, entre las dos lecturas, efectúa un reintegro e interfiere la ejecución de la transacción T2. Si las transacciones se hubieran aislado correctamente, T2 habría recuperado el mismo valor en las dos lecturas, de tal manera que, o bien habría recuperado el valor que correspondiera al saldo de la cuenta antes de que se efectuara el reintegro de T1 (es decir, habría recuperado para ambas lecturas un valor de 500€), o bien, habría recuperado el saldo que quedara después de que se efectuara el reintegro de T1 (o sea, un valor de 480€ para las dos lecturas).

Finalmente, al igual que en el caso de la lectura no confirmada, el saldo final de la cuenta es correcto.

## Análisis inconsistente (fantasmas)

| Transacción T1 (lista de cuentas)                                                                                                                                                  | Transacción T2 (creación de cuentas) |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| leer todas las cuentas del banco. Imaginemos que sólo tenemos tres cuentas (cuenta1, cuenta2 y cuenta3)<br>mostrar datos cuenta1<br>mostrar datos cuenta2<br>mostrar datos cuenta3 |                                      |
|                                                                                                                                                                                    | crear_cuenta(cuenta4)                |
|                                                                                                                                                                                    | saldo_inicial(cuenta4, 100)          |
|                                                                                                                                                                                    | COMMIT                               |
| sumar el saldo de todas las cuentas<br>obtenemos saldo cuenta1 + saldo cuenta2 + saldo cuenta3 + 100<br>(la cuenta4 con saldo 100 es el fantasma)                                  |                                      |
| COMMIT                                                                                                                                                                             |                                      |

EIMT.UOC.EDU

Los tres tipos de interferencias anteriores se producen cuando dos transacciones intentan acceder a un mismo ítem de datos y, como mínimo, una de las dos transacciones modifica ese ítem de datos. Pero también puede haber interferencias con respecto a la visión que dos transacciones tienen de un conjunto de datos que están interrelacionados. Dichas interferencias reciben el nombre genérico de análisis inconsistente. Un caso particular son los fantasmas (en inglés *phantoms*).

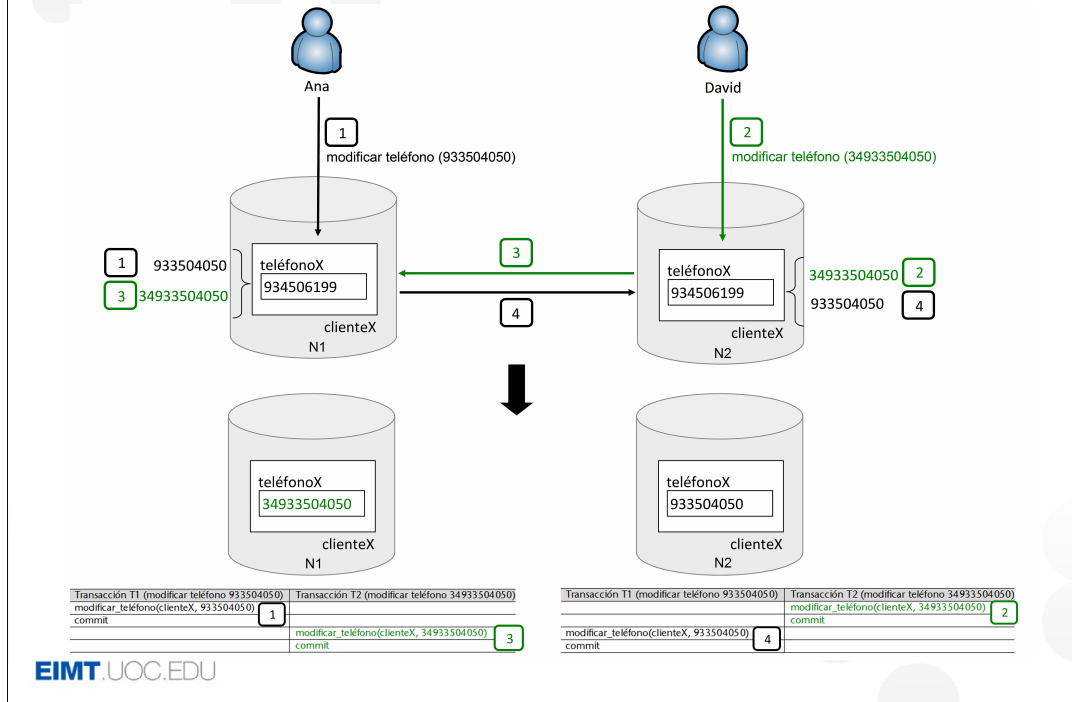
Los fantasmas se pueden producir cuando una transacción lee un conjunto de datos de interés y hay otra transacción que dinámicamente cambia dicho conjunto de datos, tal y como muestra el ejemplo.

En él podemos ver que la cuenta 4 es un fantasma desde el punto de vista de la transacción T1. Además T1 ve el efecto que tiene en la suma de saldos que se produce y que, desde su punto de vista, da un resultado incoherente. Si T1 hubiera estado aislada correctamente de la transacción T2, una vez ejecutada la primera consulta, nunca tendría que haber encontrado los datos correspondientes a la cuenta 4.

No debemos confundir los fantasmas con la lectura no repetible. Los datos leídos por la transacción T1 en relación a las cuentas 1, 2 y 3 se vuelven a recuperar con posterioridad y no se han visto alterados. El problema radica en que han aparecido nuevos datos (en concreto el saldo correspondiente a la cuenta 4).

De manera análoga a las interferencias de lectura no confirmada y lectura no repetible, la base de datos acaba en un estado consistente, dado que no se han perdido datos y éstos contienen valores correctos.

# Interferencias en BD distribuidas



En el caso de una base de datos distribuida, aparte de poderse producir las interferencias que acabamos de explicar, la existencia de réplicas puede exacerbar que las propiedades de aislamiento y consistencia se vean comprometidas, tal y como se muestra en el ejemplo.

Supongamos que la base de datos almacena información de un cliente determinado (el clienteX), y que dichos datos se encuentran replicados en dos nodos (N1 y N2). Del clienteX guardamos, entre otros datos, su teléfono y éste tiene asociado el mismo valor en ambas réplicas (934506199), tal y como se puede apreciar en la parte superior de la figura. Imaginemos que manera simultánea, dos usuarios (Ana y David) se percatan que el teléfono del clienteX es erróneo, y deciden subsanar el error. Para que ejemplo sea más interesante, el valor asignado es ligeramente diferente (si nos fijamos, David incluye el prefijo del país). Además, Ana y David tienen puntos de acceso diferentes a la base de datos distribuida (N1 para Ana y N2 para David). Nuevamente, asumimos que el gestor no realiza ningún control sobre el orden de ejecución de las operaciones. Las peticiones de Ana se indican en negro, mientras que las de David están en verde.

En primer lugar, se ejecuta en N1 la petición de cambio de Ana, de tal manera que se asigna un nuevo valor al teléfono (933504050). En segundo lugar, se procesa en N2 la petición de modificación de David, de tal manera que el teléfono del clienteX pasa a ser 34933504050. A continuación, o la aplicación o el sistema gestor de la base de datos, solicitan nuevas peticiones de cambio para mantener la consistencia de las réplicas del clienteX. En primer lugar, desde el nodo N2 se envía al nodo N1 la petición de cambio de teléfono del clienteX de acuerdo al valor especificado por David (se trata de la operación número 3), mientras que en segundo lugar desde el nodo N1 se efectúa la misma petición al nodo N2 (es la operación número 4) pero ahora en beneficio de Ana, y con el valor de teléfono que ella ha indicado.

El resultado final de la ejecución de todas estas operaciones (véase la parte inferior de la figura) es que las réplicas asociadas al clienteX no son coincidentes (en N1 el valor final corresponde a la petición formulada por David, mientras que en N2 el valor final es el indicado por Ana). Es decir, se vulnera la consistencia debido a que las transacciones no se han aislado convenientemente entre sí. En la parte inferior de la figura, se muestran las peticiones que se ejecutan en cada nodo. Parece que no hay interferencias, puesto que no existe solapamiento entre las peticiones. El problema radica en que las peticiones se han aplicado en orden diferente en cada nodo (en N1 primero se aplican los cambios de Ana y luego los de David, mientras que en N2 se hace justo a la inversa).

## Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

Seguidamente vamos a presentar los protocolos que un sistema gestor de base de datos incorpora para gestionar las transacciones. Entre ellos destacan los protocolos para el control de concurrencia que garantizan el correcto aislamiento de las transacciones y la consistencia de la base de datos ante el acceso concurrente de diversos usuarios.

## Protocolos para la gestión de transacciones

- Para mantener las propiedades de consistencia y aislamiento el SGBD implementa:
  - Protocolos de control de concurrencia
  - Protocolos para la gestión de réplicas (BD distribuidas)
- Para mantener las propiedades de atomicidad y definitividad el SGBD implementa:
  - Protocolos de recuperación: requieren de dietarios (*log*) y copias de seguridad de la BD (*backups*).
  - Protocolo de confirmación en dos fases (BD distribuidas)
- A pesar de ello, el mantenimiento de las propiedades ACID también es responsabilidad de las aplicaciones (y en consecuencia de sus desarrolladores).

EIMT.UOC.EDU

En primer lugar, el sistema gestor de la base de datos implementa protocolos para el control de concurrencia. A ellos, en el caso de una base de datos distribuida, se añaden protocolos para la gestión de réplicas. Los primeros tienen como objetivo garantizar que no se produzcan interferencias y que la consistencia de la base de datos (con independencia de que ésta sea centralizada o distribuida) no se vea vulnerada debido a la ejecución concurrente de las transacciones. Los segundos tienen como objetivo garantizar que las réplicas de unos mismos datos sean idénticas.

Por su parte, para garantizar las propiedades de atomicidad y definitividad, el gestor dispone de protocolos de recuperación, y del protocolo de confirmación en dos fases cuando la base de datos es distribuida. Los primeros impiden que en la base de datos permanezcan cambios de transacciones que no llegan a confirmar y que se pierdan los cambios que han llevado a cabo transacciones que sí han confirmado, incluso en el caso de que se produzcan averías. Para conseguir sus propósitos usan dietarios (que guardan información de las modificaciones que han efectuado las transacciones y de si éstas han confirmado o cancelado su ejecución) y copias de seguridad de la base de datos. Por su parte, el protocolo de confirmación en dos fases (o posibles variantes, como el de 3 fases) es un algoritmo que garantiza la correcta terminación de las transacciones en una base de datos distribuida. En dicho entorno, las transacciones se dividen en subtransacciones (tal y como hemos mostrado en el ejemplo de modificación del número de teléfono) que se ejecutan localmente en los diferentes nodos del sistema distribuido. Básicamente, el protocolo de confirmación en dos fases coordina que todas las subtransacciones de una transacción distribuida acaben de la misma manera (o todas confirman o todas cancelan su ejecución).

Todos estos protocolos no se ejecutan de forma aislada entre sí, sino que colaboran en la gestión de las transacciones.

Finalmente, las responsabilidades de las aplicaciones en relación al mantenimiento de las propiedades ACID incluyen: 1) la identificación de las transacciones que se necesitan ejecutar; 2) garantizar que éstas cumplen con los requerimientos formulados por los usuarios y las reglas de integridad definidas; y 3) consideraciones acerca de cómo mejorar el rendimiento de acuerdo al protocolo de control de concurrencia implementado y las posibilidades de modificar su comportamiento.



## ¿Cuándo no existen interferencias?

cuenta: 500€

| Transacción T1 (reintegro de 20) | Transacción T2 (consulta saldo)        |
|----------------------------------|----------------------------------------|
| saldo:= leer_saldo(cuenta)       |                                        |
| escribir_saldo(cuenta, saldo-20) |                                        |
|                                  | saldo:= leer_saldo(cuenta) <b>480€</b> |
| COMMIT                           |                                        |
|                                  | saldo:= leer_saldo(cuenta) <b>480€</b> |
|                                  | COMMIT                                 |

cuenta: 480€

Quando la ejecución concurrente de un conjunto de transacciones proporciona los mismos resultados que una ejecución en serie.

EIMT.UOC.EDU

cuenta: 500€

| Transacción T1 (reintegro de 20) | Transacción T2 (consulta saldo)        |
|----------------------------------|----------------------------------------|
|                                  | saldo:= leer_saldo(cuenta) <b>500€</b> |
|                                  | saldo:= leer_saldo(cuenta) <b>500€</b> |
|                                  | COMMIT                                 |
| saldo:= leer_saldo(cuenta)       |                                        |
| escribir_saldo(cuenta, saldo-20) |                                        |
| COMMIT                           |                                        |

cuenta: 480€

cuenta: 500€

| Transacción T1 (reintegro de 20) | Transacción T2 (consulta saldo)        |
|----------------------------------|----------------------------------------|
| saldo:= leer_saldo(cuenta)       |                                        |
| escribir_saldo(cuenta, saldo-20) |                                        |
| COMMIT                           |                                        |
|                                  | saldo:= leer_saldo(cuenta) <b>480€</b> |
|                                  | saldo:= leer_saldo(cuenta) <b>480€</b> |
|                                  | COMMIT                                 |

cuenta: 480€

Los protocolos para el control de concurrencia tienen como objetivo garantizar que no se produzcan interferencias y que la consistencia no se vea vulnerada debido a su existencia (pensemos, por ejemplo, en la interferencia de actualización perdida). Antes de revisarlos, es importante entender cuándo no se producen los problemas previos.

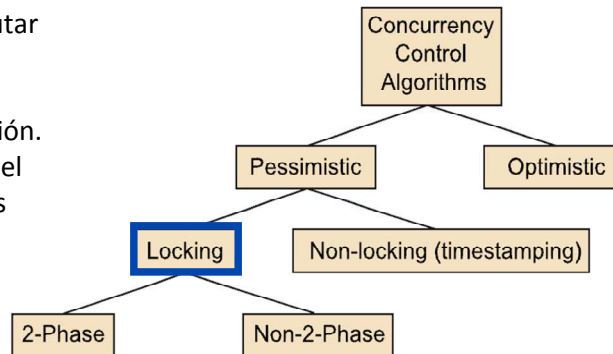
La respuesta es simple: cuando la ejecución solapada (o concurrente) del conjunto de transacciones proporcione los mismos resultados que una ejecución en serie (o en secuencia) de las mismas. En una ejecución en serie, las operaciones de una transacción no se intercalan con las peticiones de otras transacciones. Por lo tanto, es imposible que una transacción pueda interferir la ejecución de otra, o alternatively, será imposible que una transacción vea interferida su ejecución por otra transacción.

Si nos fijamos en el ejemplo, en la izquierda se muestra la ejecución concurrente de dos transacciones identificadas, respectivamente, como T1 y T2. Por su parte, en la derecha, se indican las dos ejecuciones en serie que para esas dos transacciones existen (en la parte superior, la ejecución en serie que corresponde a T2 seguida de T1, mientras que en la parte inferior se muestra T1 seguida de T2). Si nos fijamos, la ejecución solapada de T1 y T2, da los mismos resultados que la ejecución en serie T1 seguida de T2 (véase la parte inferior derecha). Por lo tanto, la ejecución concurrente de T1 y T2 es correcta, puesto que no presenta interferencias, ni tampoco los aspectos problemáticos que de ellas se pueden derivar.

A continuación pasamos a presentar las ideas clave asociadas a los principales tipos de protocolos para el control de concurrencia. La discusión, por simplicidad, se restringe a bases de datos centralizadas. A pesar de ello, los protocolos (con las adaptaciones que sean necesarias) también están disponibles en bases de datos distribuidas.

# Protocolos para el control de concurrencia

**Reservas (*locking*):** la transacción, antes de ejecutar una operación tiene que adquirir una reserva que le permita efectuar la operación. Puede causar esperas y en el caso peor abrazos mortales (*deadlocks*).



EIMT.UOC.EDU

Los protocolos para el control de concurrencia se clasifican, a un primer nivel, en dos grandes grupos, los denominados protocolos pesimistas y los optimistas. Los primeros realizan acciones correctivas para prevenir que se puedan producir interferencias. Por su parte, los protocolos optimistas no efectúan ningún control hasta la confirmación de las transacciones. En ese punto, se detecta si se han producido interferencias, y si fuese así, se ejecutan acciones correctivas.

Las reservas (en inglés *locking*) se basan en la idea que una transacción, antes efectuar una operación sobre los datos, debe adquirir una reserva sobre esos datos en una modalidad que le permita efectuar la operación que desea. Si la reserva se adquiere, la transacción pasa a efectuar la operación. En el caso contrario, la transacción bloquea su ejecución hasta que pueda adquirirla. Las reservas, si se quiere garantizar el aislamiento completo de la transacción, se liberan a su finalización (es decir, cuando la transacción ejecuta la petición de *commit* o *rollback*).

Las reservas se adquieren sobre gránulos. Un gránulo es la unidad de datos individualmente controlada por el sistema gestor de la base de datos a efectos de control de concurrencia. Su tamaño puede variar dependiendo del gestor. Por ejemplo, en una base de datos relacional, los tamaños de gránulo más frecuentes son una página de disco (que contendrá diversas filas de una tabla) o filas individuales de una tabla. Hay otras posibilidades, por ejemplo, que el gránulo sea una tabla. De hecho, en general, un mismo gestor es capaz de trabajar con diferentes niveles de granularidad (fila, página o tabla), según las necesidades.

La suspensión de la ejecución de transacciones introduce la posibilidad de que algunas transacciones queden en situación de espera indefinida (o abrazo mortal, *deadlock* en inglés). En este caso, la ejecución de dos o más transacciones queda bloqueada indefinidamente por el hecho de que, para reanudar la ejecución, todas requieren que otra de las transacciones implicadas libere alguna reserva ya obtenida. El sistema gestor de base de datos tiene que solucionar los abrazos mortales que se produzcan. En general, la situación se resuelve cancelando al menos una de las transacciones implicadas. Al cancelar una transacción, aparte de descartar los cambios que haya podido realizar, se liberan las reservas que tenía adquiridas. Esto posibilita que al menos una de las transacciones implicadas en el abrazo mortal pueda reanudar su ejecución.

# Protocolos para el control de concurrencia

**Reservas (locking):** la transacción, antes de ejecutar una operación tiene que adquirir una reserva que le permita efectuar la operación. Puede causar esperas y en el caso peor abrazos mortales (*deadlocks*).

| cuenta: 500€                                       |                                 |
|----------------------------------------------------|---------------------------------|
| Transacción T1 (reintegro de 20)                   | Transacción T2 (consulta saldo) |
|                                                    | saldo:= leer_saldo(cuenta) 500€ |
| saldo:= leer_saldo(cuenta) 500€                    |                                 |
| escribir_saldo(cuenta, saldo-20) <b>bloqueada!</b> |                                 |
|                                                    | saldo:= leer_saldo(cuenta) 500€ |
|                                                    | COMMIT                          |
| escribir_saldo(cuenta, saldo-20) 480€              |                                 |
| COMMIT                                             |                                 |
| cuenta: 480€                                       |                                 |

EIMT.UOC.EDU

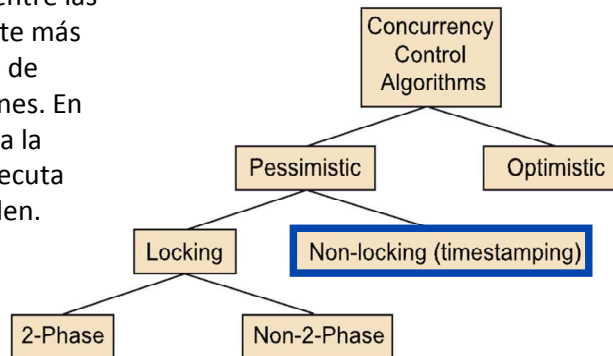
Hay distintas técnicas basadas en reservas. Éstas difieren en función de las modalidades de reserva que contemplan. La más simple (y extendida) son las reservas S, X, que incluyen dos modalidades de reserva: las compartidas (en inglés, *Shared*), o reservas S, que permiten llevar a cabo lecturas del gránulo, y las exclusivas (en inglés, *exclusive*), o reservas X, que permiten hacer lecturas y escrituras del gránulo. En esta técnica, en un momento dado, un gránulo concreto puede estar reservado por múltiples transacciones con modalidad S o bien por una sola transacción con modalidad X. En otras palabras, las reservas de modalidad S sobre un mismo gránulo son compatibles entre sí, pero las reservas X sobre un mismo gránulo son incompatibles con las reservas de tipo S o X que, sobre el mismo gránulo, ya hayan sido otorgadas.

Si aplicamos la técnica descrita a nuestro ejemplo, las transacciones T1 y T2 pueden adquirir una reserva compartida sobre el gránulo que contiene la cuenta a la que ambas desean acceder para leer su saldo (se trata de la primera operación que cada transacción efectúa). A continuación, la transacción T1 necesita modificar el saldo de la cuenta. Para ello, antes de efectuar la operación, necesita transformar la reserva sobre el gránulo de S a X. Dado que existe una transacción (en concreto la T2) que tiene una reserva con modalidad S concedida sobre ese mismo gránulo, la transacción T1 suspende su ejecución. Por lo tanto, el gestor de la base datos sólo procesará peticiones de la transacción T2. Esta transacción efectuará una nueva lectura del saldo de la cuenta corriente, y recuperará el mismo valor que en la lectura anterior (es decir, se evita la interferencia de lectura no repetible) y finaliza su ejecución. En ese momento se liberarán sus reservas y la transacción T1 podrá reanudar su ejecución, realizando el resto de operaciones que faltaban para completar el reintegro de 20€.

La idea que subyace en las técnicas de reservas (véase el ejemplo) es bloquear la ejecución de transacciones problemáticas tan pronto como surja un conflicto potencial, disminuyendo el nivel de concurrencia (y por lo tanto el rendimiento) del sistema. A partir de ese momento, las transacciones conflictivas pasan a ejecutarse en serie, o sea, sin solapar sus operaciones.

# Protocolos para el control de concurrencia

**Marcas de tiempo (*timestamping*):** impone relación de orden entre las transacciones, en su variante más simple, basadas en el inicio de ejecución de las transacciones. En caso de conflicto, se cancela la primera transacción que ejecuta una operación fuera de orden.



EIMT.UOC.EDU

Las técnicas basadas en marcas de tiempo (en inglés, *timestamping*) también son pesimistas (aunque no tanto como las reservas). Se basan en la idea de imponer una relación de orden (en concreto una relación de orden total) entre las transacciones que se ejecuten de forma concurrente. Mientras que el resultado de la ejecución sea coherente con la relación de orden preestablecida, el sistema gestor de la base de datos no efectuará ninguna acción correctiva. En el momento que el resultado no sea el deseado, la primera transacción que efectúe una operación que rompa el orden preestablecido será cancelada por el gestor (y por lo tanto, sus resultados sobre la base de datos serán descartados).

Igual que en las reservas, existen diferentes técnicas basadas en marcas de tiempo. Éstas difieren en el momento en que se establece la relación de orden entre las transacciones. En su variante más simple (el *timestamping* básico) la relación de orden se establece en el instante en que se inicia la ejecución de las transacciones.

# Protocolos para el control de concurrencia

**Marcas de tiempo (*timestamping*):** impone relación de orden entre las transacciones, en su variante más simple, basadas en el inicio de ejecución de las transacciones. En caso de conflicto, se cancela la primera transacción que ejecuta una operación fuera de orden.

| cuenta: 500€                                 |                                             |                                  |                                 |
|----------------------------------------------|---------------------------------------------|----------------------------------|---------------------------------|
| Transacción T1 (reintegro de 20)             | Transacción T2 (consulta saldo)             | Transacción T1 (reintegro de 20) | Transacción T2 (consulta saldo) |
|                                              | BoT                                         |                                  | saldo:= leer_saldo(cuenta)      |
|                                              | saldo:= leer_saldo(cuenta) <b>500€</b>      |                                  | saldo:= leer_saldo(cuenta)      |
| BoT                                          |                                             | saldo:= leer_saldo(cuenta)       | COMMIT                          |
| saldo:= leer_saldo(cuenta) <b>500€</b>       |                                             | escribir_saldo(cuenta, saldo-20) |                                 |
| escribir_saldo(cuenta, saldo-20) <b>480€</b> |                                             |                                  |                                 |
|                                              | saldo:= leer_saldo(cuenta) <b>ROLLBACK!</b> |                                  |                                 |
|                                              | <del>COMMIT</del>                           |                                  |                                 |
| COMMIT                                       |                                             |                                  |                                 |
| cuenta: 480€                                 |                                             |                                  |                                 |

Ejecución deseada por el gestor de la BD.

Vamos a aplicar la técnica de *timestamping* básico sobre nuestras transacciones de ejemplo.

Podemos ver que las transacciones incluyen un nuevo tipo de operación, denotada como BoT (*Begin of Transaction*) que el sistema gestor de la base de datos usa para determinar la relación de orden. Dado que la primera transacción que comienza es T2, ésta tendrá asignada una marca de tiempo menor al de la transacción T1. Además, el gestor controlará que la ejecución concurrente de ambas transacciones dé los mismos resultados que la ejecución en serie que viene determinada por las marcas de tiempo. Esta ejecución en serie es primero T2 y luego T1, tal y como se muestra en la parte derecha del ejemplo. Esto significa que la transacción T2, además de recuperar los mismos valores en las dos consultas, tiene que recuperar el saldo que existe en la cuenta antes de que la transacción T1 complete el reintegro de 20€.

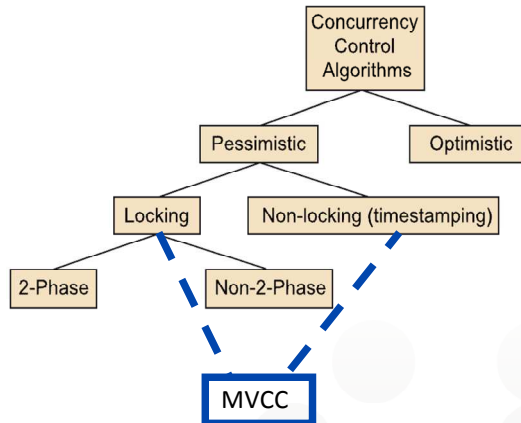
La ejecución de las transacciones transcurre con normalidad hasta que T2 intenta efectuar la segunda consulta del saldo de la cuenta. En ese momento, se rompe el orden preestablecido, dado que la transacción T2 recupera el valor dejado por T1, y eso nunca sucedería en la ejecución en serie que se desea forzar. En consecuencia, el gestor cancela la ejecución de la transacción T2, reportando algún mensaje de error a la aplicación (y por lo tanto, también al usuario). La cancelación es necesaria para evitar la interferencia de lectura no repetible.

Aunque no es el caso de nuestro ejemplo, el problema de las técnicas de *timestamping*, especialmente en su variante más simple, es que pueden causar cancelaciones innecesarias (es decir, cancelaciones de transacciones que no participan en interferencias), por el hecho que se realicen peticiones fuera de orden preestablecido.

# Protocolos para el control de concurrencia

## Mecanismo de control de concurrencia multiversión

(*multiversion concurrency control*, MVCC): enfoque híbrido. Las transacciones de sólo lectura tienen una misma visión de la BD (quizá “antigua”). Las transacciones que efectúan escrituras siguen una estrategia basada en bloqueos, y trabajan con versiones privadas que se hacen públicas a la finalización de la transacción (si ésta confirma sus resultados).



EIMT.UOC.EDU

El mecanismo de control de concurrencia multiversión (más conocido bajo el acrónimo MVCC atendiendo a su denominación en inglés) es un enfoque híbrido que combina las virtudes de las técnicas basadas en reservas y en marcas de tiempo, a la vez que minimiza sus defectos.

En esta técnica, para cada gránulo, se mantienen diversas versiones. Las versiones de un mismo gránulo reflejan las operaciones que las transacciones efectúan. Cuando una transacción quiere acceder a un gránulo, el gestor escoge la versión más adecuada a sus necesidades basándose en las marcas de tiempo que se asignan a las diferentes versiones del gránulo. Adicionalmente, cuando una transacción escribe un gránulo, si el sistema gestor de la base de datos autoriza la escritura, se crea una nueva versión del gránulo. Los usuarios (y las aplicaciones) no son conscientes de que hay múltiples versiones de un mismo gránulo. Es el gestor quien, internamente, decide qué versión de gránulo hay que escoger en beneficio de cada transacción.

Las operaciones de lectura ejecutadas por las transacciones nunca generan reservas, ni tampoco ven bloqueada su ejecución. Además, las operaciones de lectura siempre son de datos confirmados. Por su parte, las operaciones de escritura requieren la adquisición de una reserva sobre el gránulo en modalidad exclusiva (o X) por parte de la transacción que desea efectuar la operación. Esta reserva se mantiene hasta la finalización de la transacción. La adquisición de la reserva autoriza la creación de una nueva versión del gránulo que sólo será visible cuando la transacción que la crea confirme sus resultados. La creación de una nueva versión de un gránulo se hace siempre a partir de la versión del gránulo más recientemente confirmada, para evitar que se puedan perder cambios confirmados.

Dado que las operaciones de escritura necesitan reservas, las transacciones que las efectúan pueden verse involucradas en abrazos mortales que el gestor de la base de datos deberá detectar y resolver. El número de abrazos mortales, en relación a las técnicas de reservas, es potencialmente menor dado que las lecturas no requieren reservas.

El principal problema de esta técnica es que necesita una cierta capacidad de almacenamiento para absorber la creación de nuevas versiones de gránulos, y más si la base de datos se actualiza frecuentemente. Es por ello que un sistema gestor de base de datos basado en ella incorpora mecanismos para poder purgar periódicamente versiones antiguas de gránulos y liberar el espacio que éstas ocupan.

# Protocolos para el control de concurrencia

**Mecanismo de control de concurrencia multiversión** (*multiversion concurrency control, MVCC*): enfoque híbrido. Las transacciones de sólo lectura tienen una misma visión de la BD (quizá “antigua”). Las transacciones que efectúan escrituras siguen una estrategia basada en bloqueos, y trabajan con versiones privadas que se hacen públicas a la finalización de la transacción (si ésta confirma sus resultados).

| cuenta: 500€                     |                                             |                                 |      |
|----------------------------------|---------------------------------------------|---------------------------------|------|
| Transacción T1 (reintegro de 20) |                                             | Transacción T2 (consulta saldo) |      |
|                                  |                                             | saldo:= leer_saldo(cuenta)      | 500€ |
| saldo:= leer_saldo(cuenta)       | 500€                                        |                                 |      |
| escribir_saldo(cuenta, saldo-20) | 480€ copia privada!                         |                                 |      |
|                                  |                                             | saldo:= leer_saldo(cuenta)      | 500€ |
| COMMIT                           | la copia privada (480€) pasa a ser pública! |                                 |      |
|                                  |                                             | COMMIT                          |      |
|                                  |                                             | cuenta: 480€                    |      |

EIMT.UOC.EDU

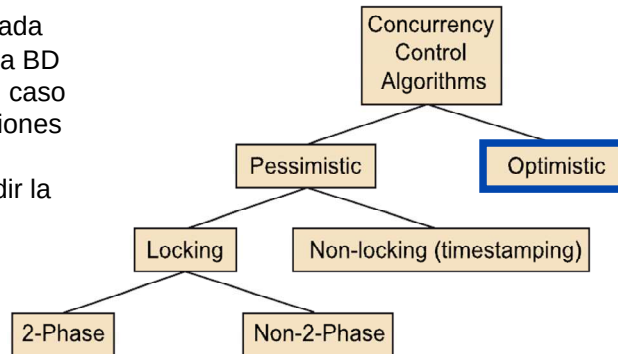
Como hemos hecho con el resto de técnicas, vamos a aplicar el mecanismo de control de concurrencia multiversión sobre nuestras transacciones de ejemplo.

Las dos lecturas que efectúa T2 devuelven el mismo valor. Este valor se corresponde a la versión de gránulo más recientemente confirmada que contenga los datos de interés (la cuenta bancaria que se desea acceder). Esta versión de gránulo ha sido creada (y confirmada) antes del inicio de la ejecución de la transacción T2 (esto se puede interpretar como que T2 accede a una versión “antigua” de la base de datos). Por su parte, la transacción T1 crea una nueva versión para el gránulo que contiene la cuenta que se desea acceder a partir de la versión leída por T2. T1 ha sido autorizada a crear la nueva versión porque no existen transacciones que intenten efectuar cambios sobre la cuenta en cuestión (en otras palabras, la transacción T1 ha podido adquirir la reserva). Esta versión constituye una copia privada, que sólo se hace pública en la base de datos cuando T1 confirma.

Fijémonos que la ejecución concurrente de T1 y T2 no presenta interferencias. Por lo tanto, debe producir los mismos resultados que una ejecución en serie. Dicha ejecución es T2 seguida de T1, tal y como se muestra a la derecha de la figura. En esencia, el sistema gestor de la base de datos ha forzado la ejecución deseada por la técnica de *timestamping*, pero evitando sus efectos negativos (gracias a la existencia de versiones de un mismo gránulo, la transacción T2 no ha sido cancelada).

# Protocolos para el control de concurrencia

**Técnicas optimistas** (*optimistic techniques*): una vez finalizada la transacción el gestor de la BD comprobará si es válida, en caso de no serlo se tomarán acciones al respecto (por ejemplo, cancelar la transacción, pedir la intervención del usuario/aplicación etc.).



Las técnicas optimistas raramente están disponibles en implementaciones comerciales. A diferencia de las pesimistas, no efectúan ningún control mientras que las transacciones están en ejecución, sino que dicho control se posterga a la finalización de las mismas. En el caso que el sistema gestor de la base de datos detecte que se han producido interferencias, pasa a tomar medidas para remediar la situación. Una medida posible es cancelar las transacciones problemáticas. En general esto no es deseable, dado que el proceso de cancelación suele ser costoso. Otra posibilidad es que el gestor delegue la decisión sobre las acciones a realizar a la aplicación o el usuario que ha solicitado la ejecución de la transacción.



## Protocolos para el control de concurrencia

- Los protocolos para el control de concurrencia más utilizados por los SGBD relacionales son las reservas y el MVCC.
- En el caso de NoSQL hay diversidad de propuestas:
  - Neo4J y MongoDB utilizan reservas.
  - CouchDB implementa el MVCC.
  - Riak utiliza *vector clocks*.

Los sistemas gestores de bases de datos relacionales, si bien inicialmente usaban como protocolo para el control de concurrencia las reservas (en general las S, X), poco a poco han evolucionado de tal manera que, a día de hoy, mayoritariamente implementan el mecanismo de control de concurrencia multiversión.

Por su parte, en el caso de la bases de datos NoSQL, existen diversidad de propuestas. A modo de ejemplo, Neo4J y MongoDB se basan en uso de reservas S, X. Por su parte, CouchDB implementa el mecanismo de control de concurrencia multiversión. Finalmente, Riak utiliza un mecanismo denominado *vector clocks* que constituye un protocolo de control de concurrencia distribuido basado en los principios de *timestamping*.

## Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

A continuación vamos a discutir algunos aspectos que, desde un punto de vista de desarrollo de aplicaciones, es necesario considerar cuando trabajamos con transacciones, tanto en el caso de bases de datos relacionales como de bases de datos NoSQL.

## Transacciones en BD relacionales

- Las BD relacionales trabajan con transacciones que verifican las propiedades ACID.
- A pesar que una transacción puede incorporar varias operaciones, el modo de trabajo por defecto acostumbra a ser que cada operación efectuada sobre la BD sea una transacción (*autocommit* activado).
- Es posible relajar el nivel de aislamiento de las transacciones, esto implica que:
  - Se pueden producir interferencias
  - Se mejora el rendimiento
  - Disminuye la sobrecarga del SGBD

EIMT.UOC.EDU

Las bases de datos relacionales trabajan con transacciones que se ajustan al modelo ACID.

A pesar que una transacción puede incorporar diversas operaciones (es decir, diversas sentencias SQL), destacar que, por defecto, se suele trabajar sobre la asunción de que cada sentencia SQL, en sí misma, constituye una transacción (esta modalidad de trabajo se conoce como *autocommit*). Si la sentencia causa cambios (por ejemplo, es un *insert*, un *update* o un *delete*) y no se produce ningún error, el sistema gestor de la base de datos confirma los resultados (es decir, ejecuta un *commit* implícito). En el caso que se produzca un error, los cambios serán automáticamente descartados (en otras palabras, el gestor ejecuta internamente una petición de *rollback*).

Para trabajar con transacciones que incorporen diversas sentencias SQL (como sería el caso de nuestros ejemplos) es necesario marcar el inicio de la transacción. Para ello SQL estándar nos proporciona la sentencia *start transaction*. La transacción permanecerá en ejecución hasta que de forma explícita indiquemos su finalización (*commit* o *rollback*).

También podemos especificar ciertas características para las transacciones (bien mediante cláusulas incluidas en la sentencia *start transaction*, bien a través de la sentencia *set transaction*). Por ejemplo, podemos indicar que la transacción sólo hará lecturas (la transacción es *read only*) o que efectuará alguna petición de cambio (en este caso se trata de una transacción *read write*). Asimismo, también podemos relajar el nivel de aislamiento de las transacciones. Esto significa que permitimos que una transacción pueda ver interferida su ejecución por otras transacciones (en definitiva, se posibilita que existan interferencias). Esto es correcto si se sabe que realmente no se producirán o si para el entorno de aplicación no es importante que se produzcan. Como efectos positivos, se mejora el rendimiento y se disminuye la sobrecarga del sistema.

A pesar de que podamos relajar el nivel de aislamiento, por defecto, hay interferencias que siempre se evitan. En concreto, y con independencia del protocolo de control de concurrencia implementado (reservas S, X o el mecanismo de control de concurrencia multiversión), la interferencia de actualización perdida siempre se evita (recordemos que ésta era la interferencia más grave que se podía producir). En el caso que el gestor implemente el mecanismo de control de concurrencia multiversión, también se evitará la interferencia de lectura no confirmada (recordemos que dicho protocolo garantiza que las lecturas siempre son de valores confirmados).

## Transacciones en BD NoSQL

- Soportar un modelo de transacciones ACID en BD que almacenan grandes volúmenes de datos, que están distribuidas y con replicación de datos es complejo y puede causar problemas de rendimiento. Es por ello que:
  - Se han propuesto modelos transaccionales alternativos (modelo BASE)
  - Se evitan transacciones que incluyan diversas operaciones (éste es el caso de las BD NoSQL basadas en modelos de agregación)
- A pesar de ello, las BD NoSQL de grafos, en general, se ajustan a modelos de transacciones ACID.

EIMT.UOC.EDU

La implementación de un modelo ACID en bases de datos altamente distribuidas y que almacenan grandes volúmenes de datos, que a su vez están replicados, es complejo. Además, el sistema resultante puede presentar un rendimiento deficiente. Esto es especialmente cierto cuando las transacciones se tienen que subdividir en subtransacciones que se ejecutan en los diferentes nodos que forman la base de datos distribuida. Garantizar que dichas subtransacciones (desde un punto de vista global) se ejecutan de manera atómica y libres de interferencias (es decir, proporcionando un resultado equivalente a una ejecución en serie), sin abrazos mortales (si se usan reservas), garantizar la correcta asignación de las marcas de tiempo (que requiere de mecanismos de sincronización) si se usa *timestamping*, o conseguir que al final de la ejecución de cada transacción todas las réplicas de unos mismos datos coincidan, es complejo y requiere de un esfuerzo considerable en términos de coordinación y comunicación entre todos los nodos. La posibilidad de que se puedan producir averías (por ejemplo, fallos en los nodos o en la red de comunicaciones), agrava aún más la situación descrita.

Por estos motivos se han propuesto modelos de transacciones alternativos, como es el caso del modelo BASE, el cual será objeto de estudio en este mismo curso. Dicho modelo se utiliza en diversas bases de datos NoSQL. El modelo BASE permite que las réplicas de unos mismos datos puedan ser, durante un período de tiempo, no coincidentes. En otras palabras, y a pesar que las transacciones hayan confirmado, se permite que la base de datos temporalmente sea inconsistente y que, por lo tanto, se puedan recuperar valores incorrectos (pérdida de aislamiento).

Otra manera de aliviar los problemas explicados, es trabajar con transacciones que únicamente incluyan una operación. Éste es el caso de las bases de datos NoSQL basadas en modelos de agregación, en donde cada operación efectuada sobre un agregado, en sí misma, es una transacción. El gestor garantizará que los cambios efectuados sobre el agregado serán confirmados en la base de datos en el caso que no se produzcan errores, o descartados en el caso contrario.

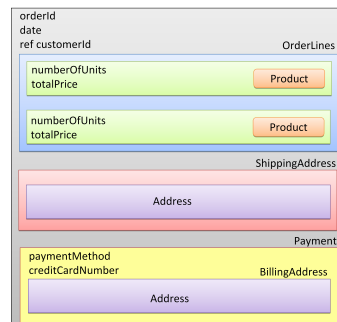
Por su parte las bases de datos NoSQL basadas en grafos, en general, acostumbran a seguir un modelo de transacciones ACID. Esto es así porque están primordialmente pensadas para trabajar en un entorno centralizado.

## Ejemplo de transacciones: el carro de la compra

```
db.order.insert({
  "orderId": 101,
  "date": "01/04/2014",
  "customerId": 1000,
  "orderLine": [
    {
      "productId": 28,
      "productName": "Principles of Distributed Database Systems",
      "numberOfUnits": 1,
      "totalPrice": 80
    },
    {
      "productId": 35,
      "productName": "Fundamentals of Transaction Processing",
      "numberOfUnits": 1,
      "totalPrice": 120
    }
  ],
  "shippingAddress": {
    "street": "Champs Élysées 156",
    "city": "Paris",
    "zipCode": "75008",
    "country": "France"
  },
  "payment": {
    "paymentMethod": "AVISA",
    "creditCardNumber": "2345-234-444",
    "billingAddress": {
      "street": "Champs Élysées 156",
      "city": "Paris",
      "zipCode": "75008",
      "country": "France"
    }
  }
})
```

Inserción de un nuevo pedido

EIMT.UOC.EDU



```
Payment(paymentId, paymentMethod, creditCardNumber, billingAddressId)
(billingAddressId) clave foránea a Address

Order(orderId, date, customerId, paymentId, shippingAddressId)
(customerId) clave foránea a Customer
(paymentId) clave foránea a Payment
(shippingAddressId) clave foránea a Address

OrderLine(orderLineId, orderId, productId, numberOfUnits, totalPrice)
(orderId, productId) clave alternativa
(orderId) clave foránea a Order
(productId) clave foránea a Product

start transaction read write, isolation level serializable;
insert into payment values(11, "AVISA", "2345-234-444", 201);
insert into order values(101, "01/04/2014", 1000, 11, 201);
insert into orderLine values(1, 101, 28, 1, 80);
insert into orderLine values(2, 101, 35, 1, 120);
commit;
```

A pesar de que las bases de datos NoSQL basadas en modelos de agregación no permitan definir transacciones que incluyan más de una operación ni garantizar transacciones ACID, el diseño de agregados puede ayudar a simular prestaciones similares, tal y como ilustra el ejemplo.

En él se muestra el diseño de un agregado para el dominio de aplicación del carro de la compra que ya hemos utilizado en otras ocasiones. El agregado (véase la parte superior derecha de la figura) representa un pedido e incluye datos generales (por ejemplo quién y cuándo se efectúa el pedido), las líneas de pedido, la dirección de envío y los datos de pago.

En la parte de la izquierda se muestra la inserción de un nuevo pedido en MongoDB (una de las bases de datos NoSQL de documentos más utilizadas). En concreto, se está insertando un nuevo pedido en una colección de pedidos (*db.order.insert*). En MongoDB, el concepto de colección es equiparable al de tabla en una base de datos relacional, y sirve para almacenar objetos de un mismo tipo. En el caso de que no se produzcan errores, MongoDB garantiza que los cambios producidos (la inserción de todos los datos relativos al pedido) serán definitivos en la base de datos.

Por otro lado, en el rectángulo de la parte inferior derecha, se muestra cómo se realizaría la inserción de todos los datos relativos a un nuevo pedido en el caso de una base de datos relacional. Además del esquema asociado a las tablas implicadas (la tabla de pago, pedido y línea de pedido), se muestra la transacción que inserta un nuevo pedido. Esta transacción constituye la unidad atómica de ejecución y los resultados de la misma se harán definitivos en la base de datos cuando la transacción confirme. Fijémonos que se trata de una transacción *read write* y que no se permite, bajo ningún concepto, que la transacción vea interferida su ejecución (nivel de aislamiento *serializable*).

## Bases de datos distribuidas

- Consistencia
- Transacciones y propiedades ACID
- Protocolos para la gestión de transacciones
- Transacciones en BD relacionales y BD NoSQL

En esta presentación hemos explicado las posibles anomalías que se derivan del acceso simultáneo de diversos usuarios a la base de datos. Para solventarlas, los sistemas gestores de bases de datos se fundamentan en transacciones y en una serie de mecanismos que las gestionan. Existen diversos modelos de transacciones. Nuestro foco de interés ha sido el modelo ACID.

## Referencias

A. Abelló (2012). Transaction Models and Concurrency Control. Material docente UOC, asignatura Arquitectura de bases de datos.

P.A. Bernstein & E. Newcomer (2009). *Principles of Transaction Processing (2nd Edition)*. Morgan Kaufmann.

Joe Celko's (2013). *Complete Guide to NoSQL*. Elsevier.  
(<http://www.sciencedirect.com/science/book/9780124071926>)

L. Liu & M.T. Özsu (Eds.) (2009). *Encyclopedia of Database Systems*. Springer.

M.T. Özsu & P. Valduriez (2011). *Principles of Distributed Systems*. 3rd edition. Springer.

M.E. Rodríguez (2011). Gestión de transacciones. Material docente UOC, asignatura Uso de bases de datos.

P.J. Sadalage & M. Fowler. (2013). *NoSQL Distilled. A brief Guide to the Emerging World of Polyglot Persistence*, Pearson Education.

EIMT.UOC.EDU

Esperamos que hayáis disfrutado y aprendido con este vídeo. A continuación encontraréis algunas referencias que os permitirán profundizar más en los conceptos que hemos tratado.

Que tengáis un buen día.